

Phase 15: Streamlit UI

Hospital Patient Helpdesk Chatbot

Python module: 08_app/15_streamlit_app.py

Jupyter notebook: 13_notebooks/15_streamlit_app.ipynb

Purpose: Create a web-based chatbot interface for patients.

1. Phase objective

Phase 15 provides the patient-facing Streamlit interface for the Hospital Patient Helpdesk Chatbot. It uses the same Phase 12 ChatService that powers the API, so UI behavior stays aligned with retrieval, prompting, generation, and Phase 11 guardrails.

The module can run in two modes:

- streamlit run 08_app/15_streamlit_app.py renders the web UI.
- python 08_app/15_streamlit_app.py runs a deterministic UI evaluation and writes 15_ artifacts.

2. Folder placement

```
hospital_patient_helpdesk_chatbot/  
|-- 01_data/  
|   |-- processed/  
|       |-- 15_streamlit_transcript.json  
|       |-- 15_streamlit_session_report.json  
|       |-- 15_streamlit_ui_audit.csv  
|       |-- 15_failed_streamlit_turns.json  
|       |-- plots/  
|           |-- 15_streamlit_response_modes.png  
|           |-- 15_streamlit_safety_actions.png  
|-- 07_backend/  
|   |-- 12_api_main.py  
|-- 08_app/  
|   |-- 15_streamlit_app.py  
|   |-- 15_streamlit_app.md  
|   |-- 15_streamlit_app.pdf  
|   |-- assets/  
|       |-- hospital_logo.png  
|-- 12_tests/  
|   |-- test_streamlit_app.py  
|-- 13_notebooks/  
|   |-- 15_streamlit_app.ipynb  
|-- README.md
```

All generated artifacts and plots begin with 15_.

3. Inputs

Input	Purpose
07_backend/12_api_main.py	Provides ChatService, request validation, and guarded responses.
08_app/assets/hospital_logo.png	Optional logo displayed in the Streamlit header.
05_vector_store/chroma_db/06_vector_index.sqlite3	Used indirectly by the backend service.
02_config/prompt_config.yaml	Used indirectly by prompt construction.
.env	Provider/model configuration. The default remains offline.

4. Output files

Numbered artifact	Description
15_streamlit_transcript.json	Sample chat turns with question, response, timestamp, and metadata.
15_streamlit_session_report.json	Mode counts, safety-action counts, latency summary, and configuration.
15_streamlit_ui_audit.csv	Compact transcript rows for review.
15_failed_streamlit_turns.json	Sanitized failed sample turns.
15_streamlit_response_modes.png	UI response-mode counts.
15_streamlit_safety_actions.png	UI guardrail-action counts.

The plots describe the sample UI flow and do not measure clinical correctness.

5. UI features

- Header with hospital logo when available.
- Clear emergency warning above the chat box.
- Example-question buttons in the sidebar.
- Streamlit chat messages for user and assistant turns.
- Grounded answer text with confidence and source badges.
- Guardrail status line for every assistant response.
- Optional thumbs feedback widget.
- Conversation audit JSON download.
- Clear-conversation button.

6. Python module code sections

Configuration and contracts

`StreamlitUIConfig` stores UI title, page icon, provider, visible source count, and feedback setting. `ChatTurn` stores one user/assistant turn. `UIEvaluationResult` records generated artifacts and totals.

Project and backend loading

`default_project_root()`, `import_module()`, and `load_api_module()` locate and import the Phase 12 backend module by path.

Rendering helpers

`source_badges()` formats compact citation/source labels. `safety_banner()` converts guardrail actions and risk levels into patient-facing status text. `response_to_markdown()` renders the assistant answer, confidence, safety status, sources, and triggered rules. `transcript_rows()` flattens turns for audit output.

Backend service helpers

`create_chat_service()` initializes the Phase 12 `ChatService`. `ask_service()` normalizes a question and sends it to the shared backend service.

Evaluation artifacts

`run_ui_evaluation()` simulates three patient questions: two grounded questions and one emergency question. It writes transcript, report, audit, failure, and plot artifacts.

Streamlit runtime

`render_streamlit_app()` creates the real Streamlit UI. `running_in_streamlit()` avoids importing Streamlit during bare Python evaluation. `main()` renders the app when launched by Streamlit and otherwise runs artifact generation.

7. Notebook code sections

The notebook:

1. locates the project from workspace, project root, or notebook folder;
2. imports the shared Streamlit module;
3. verifies helper formatting;
4. creates the backend chat service;
5. tests one grounded answer and one emergency override;
6. runs the full UI evaluation;
7. validates generated artifacts; and
8. displays both numbered plots.

8. Notebook versus Python module

Topic	15_streamlit_app.ipynb	15_streamlit_app.py
Main purpose	Interactive validation of UI helpers and sample chat flow.	Reusable Streamlit app plus CLI artifact generation.
Web rendering	Does not launch a web server.	Renders the Streamlit UI when launched with <code>streamlit run</code> .
Backend	Calls the same ChatService.	Owens service loading and UI request handling.
Outputs	Displays examples, report, and plots inline.	Writes 15_ JSON, CSV, and PNG artifacts.
Testing	Uses helper assertions.	Provides functions covered by <code>test_streamlit_app.py</code> .

9. Running the app

From the workspace root:

```
streamlit run hospital_patient_helpdesk_chatbot/08_app/15_streamlit_app.py
```

Generate Phase 15 artifacts without opening the UI:

```
python hospital_patient_helpdesk_chatbot/08_app/15_streamlit_app.py
```

10. Automated tests

12_tests/test_streamlit_app.py covers:

- compact source badges;
- safety banners for pass and override actions;
- markdown rendering with sources;
- transcript flattening; and
- Streamlit dependency availability.

In the current interpreter, pytest is not installed, so direct module and notebook validation were used.

11. Validation results

The included UI evaluation produced:

- 3 input questions;
- 3 transcript turns;
- 0 failed turns;

- 2 grounded answers;
- 1 emergency override;
- 2 pass safety actions;
- 1 override safety action; and
- `streamlit_available` recorded as a boolean environment check.

12. Safety and deployment notes

- Display emergency guidance prominently.
- Do not present the UI as emergency care or clinical decision support.
- Keep source citations visible for grounded answers.
- Avoid storing protected health information in downloadable transcripts.
- Add authentication, access controls, logging policies, rate limits, and privacy review before deployment.
- Continue using Phase 11 guardrails and backend monitoring for every patient-facing response.